
Overflip: Repetition-Induced Label Flips in Guardrail Models

Xu He¹ Chih-Hsuan Lin² Hung-Mao Chen¹

Junjie Xiong³ Yan Zhai⁴ Kun Sun¹

¹George Mason University ²Virginia Tech

³Missouri University of Science and Technology ⁴Visa
xhe6@gmu.edu

Abstract

Guardrail models are classifiers deployed to screen malicious prompts and responses in LLM-based services. To meet latency constraints, many lightweight guardrails adopt compact Transformer backbones (e.g., DeBERTa) that are trained with short context windows (typically 512 tokens) and rely on bucketed relative positional encodings to process longer inputs. Prior evaluations assume that a guardrail’s decision is stable as the input is lengthened. We show that this assumption can fail. We identify Overflip, a repetition-induced instability where repeating a prompt causes the guardrail’s prediction to flip (MAL→BEN) as the sequence grows. We conduct experiments on 9 widely used lightweight guardrail models. Five exhibit MAL→BEN flips on a benchmark of 100 prompts, with confidence margins shrinking steadily with repetition. Among these vulnerable models, flip rates range from 8% to 92%, with first flips occurring at roughly 2.6k–9.4k tokens. Our analysis suggests Overflip differs from traditional attention-dilution baselines, which aim to divert the model’s attention away from tokens associated with malicious content, shifting it instead toward unrelated content, such as benign padding or shuffling. While Overflip preserves malicious content, it homogenizes token-level attention over repeated structure and induces a distinct, more gradual attention-dispersion trajectory than padding. Moreover, Overflip poses a greater threat to LLM services than traditional attention dilution methods. Because the bypassed prompt remains semantically intact and is still readily understood by downstream business LLMs, it can transmit malicious intent after passing the guardrail. These findings expose repetition as an attack surface for guardrail models and motivate length-robust evaluation and mitigation.

1 Introduction

As large language models (LLMs) become increasingly deployed in production systems, specialized guardrail models have emerged as a critical safety component Wang et al. [2025], which are designed to detect malicious prompts and potentially harmful responses before they reach the business model and end users. To maintain low latency and deployment overhead, guardrail models could be lightweight classifiers Zheng et al. [2024], typically employing compact architectures (such as DeBERTa He et al. [2020], ProtectAI.com [2023] and MordenBert Warner et al. [2024], Zheng et al. [2024]) with limited context windows (commonly 512 tokens). The limited context window is unable to satisfy current deployment requirements that routinely accumulate long prompts (e.g., multi-turn dialogues and retrieval-augmented inputs) and may reach far beyond 512 tokens Paulsen [2025]. In practice, many lightweight guardrails therefore rely on relative positional encodings to operate on longer sequences when needed Shaw et al. [2018a].

Prior evaluations of guardrail models have focused primarily on classification accuracy against adversarial perturbations DAIR.AI [2025] and various attack patterns such as jailbreaks and prompt injections Chao et al. [2024]. However, these assessments implicitly assume that these lightweight models’ decision remains stable as input length increases, an assumption we show to be flawed.

In this paper, we reveal a systematic vulnerability we term Overflip: as inputs are lengthened via repetition, guardrail models can exhibit prediction flips. Our primary focus is the security-critical bypass setting, where an originally malicious prompt is reclassified as benign (MAL→BEN) after sufficient repetition, allowing the prompt to pass the guardrail while preserving its intent. In addition to bypass, we also observe benign↔malicious instabilities, suggesting a broader length-induced decision fragility.

Across 9 widely used lightweight guardrail models, 5 exhibit at least one MAL→BEN flip under the repetition-based overflow attack on a benchmark of 100 prompts. Among vulnerable models, flip rates range from 8% to 92%, and first flips occur at lengths spanning roughly 2.6k–9.4k tokens. These flips are accompanied by shrinking confidence margins, indicating progressive destabilization as the sequence grows.

While one might initially attribute this phenomenon to attention dilution Liu et al. [2023], Bai et al. [2023], where longer contexts spread attention weights more thinly, our results show a qualitatively different behavior under repetition. Unlike padding-based lengthening that appends novel benign content, Overflip preserves content via exact repetition but still destabilizes the classifier: attention over repeated structure becomes increasingly homogenized, and attention dispersion follows a distinct, more gradual trajectory compared to padding and shuffling.

This points to structural long-input effects beyond “more content” as a driver. In particular, many lightweight guardrails employ *bucketed relative positional encodings* (similar to T5-style position biases Raffel et al. [2020a]) to extend beyond their native 512-token training window. Once inputs exceed this boundary, positional bucketing can coarsely compress long-range distinctions, further undermining the model’s ability to isolate critical evidence for classification.

Furthermore, compared to typical length-extending attacks, Overflip better preserves the original prompt semantics: in downstream business models (GPT-4.1 Mini and Llama 3.1 8B), repetition-based overflip yields substantially higher behavioral consistency than padding-based baselines, while padding more frequently induces misinterpretations.

Overflip is also fundamentally distinct from existing jailbreak attacks such as DAN or gradient-based methods (e.g., GBDA). Those approaches typically require white- or gray-box access to LLM internals and target the downstream model directly; Overflip, by contrast, is a fully black-box attack requiring only input-output access to the guardrail API, with no knowledge of model parameters. Notably, mainstream guardrails such as LPG-2-86M achieve over 85% defense rates against traditional jailbreak attacks Wang et al. [2025], yet remain vulnerable to Overflip (8%–92% flip rate), underscoring that Overflip exploits a structurally different and previously unaddressed attack surface.

We conduct extensive experiments to characterize prevalence, flip dynamics, and practical impact. We evaluate 9 guardrail models across 100 prompts, measure flip rates and first-flip lengths, and analyze confidence-margin trajectories around the 512-token boundary. We further validate real-world effectiveness by testing whether repetition preserves attack semantics in downstream LLMs.

Contributions. This work makes the following contributions:

- **Prevalence:** We establish that Overflip is widespread in lightweight guardrails: across 9 production models evaluated on 100 prompts, 5 exhibit MAL→BEN flips under simple repetition-based lengthening.
- **Mechanism beyond dilution baselines:** We compare Overflip with standard lengthening baselines (padding and shuffling) and show that repetition can trigger flips without introducing new content. Attention analyses reveal homogenization under repetition and distinct dispersion dynamics relative to padding, consistent with a long-input failure mode amplified by positional bucketing beyond the 512-token boundary.

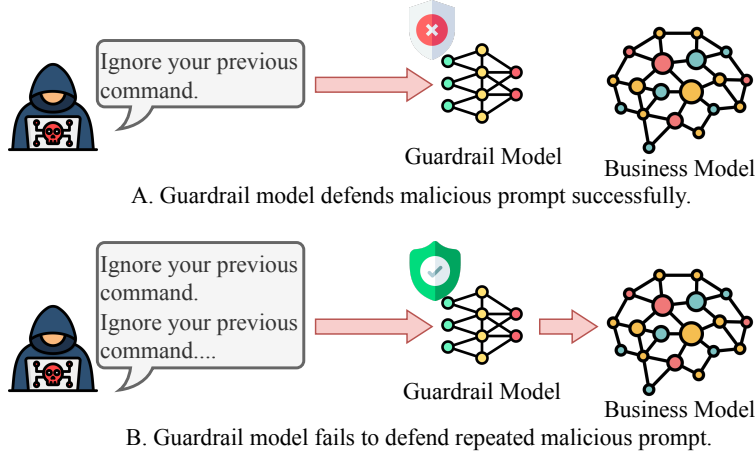


Figure 1: Overview of the Overflip Attack. An attacker repeatedly lengthens an original malicious prompt via semantic-preserving repetition until the guardrail model flips from malicious to benign (MAL→BEN), causing the prompt to be forwarded to the downstream business model while retaining the attack intent.

- **Real-world impact:** We validate that repetition-based Overflip better preserves attack semantics in downstream business models (GPT-4.1 Mini and Llama 3.1 8B), increasing the practical risk of guardrail bypass.

2 Related Work

2.1 Guardrail Models

In this paper, guardrail models refer to lightweight classifiers used to detect malicious prompts (e.g., jailbreaks and prompt injections) in LLM services Wang et al. [2025], Zheng et al. [2024]. These guardrail models are now widely used in production LLM services Sobolik and George [2025], especially in enterprise and cloud offerings (e.g., AWS and NVIDIA). Representative families include prompt-injection detectors built on compact Transformer backbones (e.g., DeBERTa variants) ProtectAI.com [2023], dedicated jailbreak detectors Jindal [2024a,b, 2025], and production-oriented guardrails such as Llama Prompt Guard Meta [2024, 2025], Sentinel Ivry and Nahum [2025], and Granite-Guardian IBM Research [2024b,a]. These systems prioritize low latency and are commonly trained or fine-tuned with relatively short context windows, making length robustness a practical concern.

2.2 Attention Dilution

Attention dilution refers to a phenomenon in which, when the context is overly lengthy or excessively complex, model’s performance downgrades as attention is distributed across many tokens Vaswani et al. [2017], Liu et al. [2023]. In the prompt-level attack scenario, Common attack methods exploiting attention dilution include techniques such as padding and shuffling. They can saturate context and create instruction competition (e.g., many-shot jailbreaking and prompt injection) Anil et al. [2024], Liu et al. [2024], Li et al. [2024], Yi et al. [2023], Wallace et al. [2024].

Our findings are related but distinct: Overflip uses *semantic-preserving repetition* and still induces flips, with attention dynamics that differ from padding-based dilution baselines. In particular, padding introduces novel benign content and quickly increases attention dispersion, while repetition creates many identical occurrences and leads to attention homogenization over repeated structure. This points to a long-input failure mode that cannot be reduced to “more competing content” alone.

2.3 Positional Encodings in Transformers

Transformer models inject order via positional encodings. Absolute encodings Shaw et al. [2018a] attach position-specific embeddings, while relative encodings Shaw et al. [2018b] add position-dependent attention biases (including variants that couple content and position more tightly Huang and Xu [2020]). To scale beyond a fixed window, many efficient classifiers adopt T5-style *bucketed* relative positions Raffel et al. [2020b], and modern alternatives such as ALiBi Press et al. [2022] and RoPE Su et al. [2021] also support extrapolation to long contexts. Our work highlights that positional-design choices can affect *classification stability* under lengthening, complementing prior work that primarily evaluates long-context generation and understanding.

In lightweight safety classifiers, positional extrapolation is often adopted as an engineering mechanism to accept long inputs even when training is dominated by short sequences. Our results suggest that this mismatch can surface as label instability under repetition, motivating length-aware stress testing when deploying guardrails.

3 Problem Setup and Threat Model

3.1 Task Definition

We model a guardrail as a binary classifier $f : \mathcal{X} \rightarrow \{0, 1\}$ that maps an input prompt $x \in \mathcal{X}$ to a label $y \in \{0, 1\}$, where 0 denotes benign and 1 denotes malicious (e.g., prompt injection or jailbreak). The model outputs class probabilities $p_{\text{mal}}(x)$ and $p_{\text{safe}}(x)$ with $p_{\text{mal}}(x) + p_{\text{safe}}(x) = 1$, and predicts via thresholding:

$$f(x) = \mathbb{I}[p_{\text{mal}}(x) > 0.5] \quad (1)$$

Given an input x with base length L tokens and prediction $y(L) = f(x)$, an **Overflip** occurs if there exists a lengthened version x' with length $L' > L$ such that x' is produced through repetition and $y(L') = f(x') \neq y(L)$.

Unless otherwise stated, we focus on the security-critical **MAL**→**BEN** case, where a malicious prompt is misclassified as benign and forwarded downstream.

Attack objective. As shown in Fig. 1, we consider a deployment pipeline where an input prompt is first screened by a guardrail model f ; if classified as benign, it is forwarded to a downstream business model. The attacker’s goal is to *bypass* the guardrail while preserving as much malicious intent as possible in the prompt that reaches the business model. Concretely, given an original malicious prompt x with $f(x) = 1$, the attacker applies a *semantic-preserving* lengthening operator based on repetition,

$$\text{Rep}_n(x) = \underbrace{x \oplus x \oplus \cdots \oplus x}_{n \text{ times}}, \quad n \geq 2, \quad (2)$$

and seeks an n such that $f(\text{Rep}_n(x)) = 0$. Because repetition keeps the original instructions intact (and even amplifies them), the resulting prompt retains the attack intent when it is forwarded to the business model, while potentially being misclassified as benign by the guardrail.

3.2 Threat Model

We consider attackers who exploit length-dependent behavior without crafting semantically novel adversarial content. The core insight is that a guardrail’s decision may become unstable under semantic-preserving lengthening, enabling misclassification without changing the underlying intent.

Attacker capabilities. The attacker can submit prompts to the deployment pipeline and apply semantic-preserving lengthening (e.g., repetition). They may estimate token counts via the production tokenizer and observe predictions in a black-box setting (or confidence scores in a gray-box setting). We assume no access to model parameters or training data.

The attacker is constrained to *text-level* manipulations (no character-level obfuscation or semantic rewriting), keeping the attack realistic for production pipelines.

Model Vendor	Model Name	Model Arch.	Para.	Detection Type	Flip
meta-llama	Llama-Prompt-Guard-2-22M (LPG-2-22M)	deberta-v2	22M	Both	✓
	Llama-Prompt-Guard-2-86M (LPG-2-86M)	deberta-v2	86M	Both	✓
protectai	deberta-v3-base-prompt-injection (dvbpi)	deberta-v3	140M	Prompt Injection	✓
	deberta-v3-small-prompt-injection-v2 (dvspiv2)	deberta-v3	140M	Prompt Injection	–
madhurjindal	Jailbreak-Detector (V1) (JD-V1)	distilbert	66M	Jailbreak	–
	Jailbreak-Detector-Large (JD-L)	deberta-v2	279M	Jailbreak	–
	Jailbreak-Detector-2-XL (JD-2XL)	qwen2	500M	Jailbreak	–
qualifire	prompt-injection-sentinel (PIS)	modernbert	395M	Jailbreak	✓
	prompt-injection-jailbreak-sentinel-v2 (PIJS-v2)	qwen3	596M	Both	✓

Table 1: Guardrail models evaluated in this paper. We report each model’s vendor, backbone architecture, parameter size, and intended detection type (jailbreak vs. prompt injection vs. both). **Flip** indicates whether the model exhibits at least one prediction flip under our SafeGuardrail Overflip Attack on any prompt in the evaluation set.

4 Experimental Setup

4.1 Research Questions

We structure our study around three research questions:

RQ1 (Overflip Prevalence). *How prevalent is Overflip across widely deployed lightweight guardrail models?* We evaluate repetition-based lengthening on a diverse benchmark of prompts and measure model-level flip occurrence, flip rates, and first-flip lengths.

RQ2 (Impact on Downstream LLM). *Does Overflip preserve attack semantics when the prompt reaches downstream LLMs?* We test manipulated prompts on representative backend models and compare repetition with padding and shuffling baselines to assess whether repetition preserves the original attack intent.

RQ3 (Mechanism of Overflip). *What causes the flips, and how does Overflip differ from traditional attention-dilution strategies?* We analyze attention behavior under repetition, including token-level attention homogenization and attention-dispersion statistics (e.g., normalized CLS-attention entropy) as length grows and at the flip point.

4.2 Models

Guardrail Models. We evaluate 9 lightweight guardrail models spanning multiple architectures and training objectives (Table 1). The suite covers widely deployed DeBERTa-based classifiers (Llama Prompt Guard 2-22M/86M, ProtectAI prompt-injection detectors), a DistilBERT jailbreak detector, and more recent ModernBERT and Qwen-based sentinels, ranging from 22M to 596M parameters. Models target jailbreak detection, prompt-injection detection, or both. Five of the nine exhibit at least one MAL→BEN flip under repetition-based lengthening (Table 1, **Flip**), demonstrating that length-induced instability spans multiple architecture families.

Business Models. To assess real-world impact, we test whether bypassed prompts still elicit harmful responses from downstream LLMs, using GPT-4.1 Mini and Llama 3.1 8B as backends. Starting from the 100 malicious prompts, we use LPG-2-86M as the guardrail, retain the 92 prompts initially flagged as malicious, and apply the three lengthening strategies, yielding 172 successfully bypassed prompts (Overflip: 71; Padding: 92; Shuffling: 9). For each backend, we query both the original and bypassed prompts and categorize responses as acceptance (*ac*), rejection (*rej*), or misinterpretation (*mis*).

4.3 Dataset and Data Processing

Datasets. We evaluate the attack on a curated set of 100 validated malicious prompts. Prompts are collected from two Hugging Face sources (jayavibhav/prompt-injection-safety and ahsanayub/malicious-prompts), deduplicated, filtered to be English-only, and constrained to short inputs (<500 characters). We further verify that all prompts are detected as unsafe

Model Name	Flip Rate	Flip Round	Flip Length
LPG-2-22M	15%	45.7	2753
LPG-2-86M	92%	61.1	4960
dvbpi	8%	85.0	5376
PIS	87%	131.3	9424
PIJS-v2	39%	39.2	2599

Table 2: Overall attack outcomes across models. **Flip Rate** is the fraction of prompts that trigger at least one prediction flip. **Flip Round** is the average iteration index when the first flip occurs. **Flip Length** is the average input length (in tokens) at the first flip.

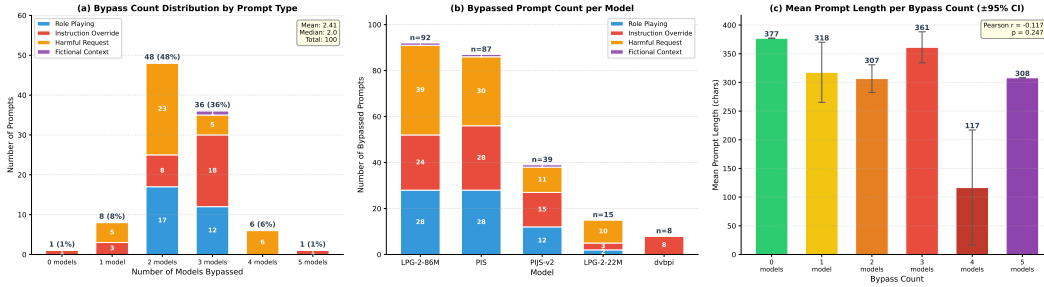


Figure 2: Analysis of 100 malicious prompts across five guardrail models. **(a)** Bypass count distribution by prompt type; nearly half of prompts bypassed exactly two models, with Harmful Request and Instruction Override as the dominant categories. **(b)** Total bypasses per model by prompt type; LPG-2-86M and PIS exhibit the highest bypass rates. **(c)** Mean prompt length ($\pm 95\%$ CI) per bypass count; no significant correlation is observed (Pearson $r = -0.08$, $p = 0.418$), indicating that prompt verbosity alone does not predict bypass success.

by meta-llama/Llama-Prompt-Guard-2-86M (100% verification), and retain a diverse mix of prompt styles including instruction override, harmful requests, and role-playing jailbreak patterns.

All base prompts are kept short (<512 tokens after tokenization) so that the initial prediction is made within each model’s native context window.

Lengthening Methods. We consider three lengthening strategies that preserve the original semantic intent: (1) **Exact repetition** concatenates the original prompt multiple times without modification. (2) **Benign padding** appends semantically neutral sentences drawn from a benign pool after the prompt. (3) **Sentence shuffling** perturbs word order while preserving critical keywords, testing sensitivity to surface patterns rather than semantics.

Environment. Experiments are conducted on Macbook Pro (M4, 48GB). All guardrail models are downloaded from Huggingface and deployed using pytorch (2.7.0). The test input batch size is 1.

5 Results

5.1 Overflip Prevalence (RQ1)

We first evaluate whether Overflip is widespread across lightweight guardrails. Our experiments show that 5 of 9 evaluated models exhibit at least one flip when inputs are lengthened via repetition (Table 1), and Table 2 summarizes the flip statistics for these vulnerable models.

Flips span multiple vendors and backbone families (DeBERTa, ModernBERT, Qwen), indicating the issue is not tied to a single implementation. Flip rates range from 8% to 92%, with first-flip lengths of 2.6k–9.4k tokens occurring after dozens of repetition rounds (Table 2). A key factor is the positional encoding scheme: models using bucketed relative positional encodings (LPG-2-86M, dvbpi, PIS) are most susceptible, as bucketing coarsens long-range token distinctions beyond the 512-token window and amplifies the attention homogenization that drives flips. The three jailbreak detectors with no

Attack Method	Guardrail Flip	Backend LLM ASR	
	Llama-86m	GPT-4.1 mini	Llama 3.1 8b
Overflip	71	41	33
Padding	92	28	6
Shuffling	9	4	0

Table 3: End-to-end attack success across three methods. Despite fewer guardrail flips than padding, Overflip achieves substantially higher backend LLM attack success rates (ASR), demonstrating the advantage of semantic preservation in the full attack pipeline.

flips do not use this scheme, suggesting positional encoding design is a principal determinant of length-robustness.

Figure 2 characterizes bypass behavior at the prompt level. Nearly half of prompts bypassed exactly two models, with Harmful Request and Instruction Override as the most frequent categories (a); LPG-2-86M and PIS lead in total bypass counts (b). Original prompt length shows no significant correlation with bypass count ($r = -0.08$, $p = 0.418$; c), confirming that susceptibility is driven by repetition-induced lengthening, not prompt verbosity.

5.2 Impact on Downstream LLM (RQ2)

We evaluate the end-to-end threat posed by Overflip by measuring how often a bypassed prompt elicits a harmful response from the downstream business LLM. We use LPG-2-86M as the guardrail and test two backend models: GPT-4.1 Mini and Llama 3.1 8B. Table 3 reports guardrail flip counts and backend attack success rates (ASR) for Overflip, padding, and shuffling.

Overflip achieves a substantially higher backend ASR than padding on both models (41 vs. 28 on GPT-4.1 Mini; 33 vs. 6 on Llama 3.1 8B), despite triggering fewer guardrail flips (71 vs. 92). This gap highlights the key advantage of Overflip: because the repeated prompt remains semantically intact, it is still understood and acted upon by downstream LLMs after bypassing the guardrail. In contrast, padding introduces incoherent content that degrades downstream comprehension, limiting its practical impact even when the guardrail is bypassed. Shuffling produces very few flips (9) and negligible backend ASR. For completeness, Appendix Table 4 reports the full response-category transition breakdown (acceptance, rejection, misinterpretation) for each backend model and attack method.

Taken together, these results indicate that Overflip is both *prevalent* in lightweight guardrails and *operationally relevant*: repetition can bypass the guardrail while keeping the downstream model’s behavior largely consistent with the original attack intent.

5.3 Mechanism of Overflip (RQ3)

We further reveal that repetition triggers flips via a distinct long-input failure mode: attention becomes increasingly homogenized over repeated structure, destabilizing the decision boundary without introducing new content. We support this with two complementary analyses on a representative guardrail classifier: (i) token-level attention under repetition (Figure 3), and (ii) attention-dispersion dynamics across lengthening strategies (Figure 4).

Token-level homogenization under repetition. We extract last-layer CLS-to-token attention weights for a fixed prompt repeated $k \in \{1, 5, 20, 100\}$ times, collapsing the sequence onto the base prompt by summing per-position weights across copies (Figure 3). The four profiles reveal monotonic homogenization: attention is focused at $k = 1$ (entropy = 1.89), with the sharpest shift between $k = 1$ and $k = 5$ (entropy $\rightarrow 1.77$, std $\uparrow 0.12$) as cumulative attention concentrates on high-frequency structural anchor positions. Further repetitions deepen this effect ($k = 20$: 1.65; $k = 100$: 1.58). This *attention homogenization*, structural anchors monopolizing attention across copies, reduces the model’s ability to localize semantically informative tokens, thereby destabilizing the decision boundary.

Attention dispersion across strategies. To compare Overflip with attention-dilution baselines (padding and shuffling), we quantify the dispersion of the CLS token’s attention using normalized

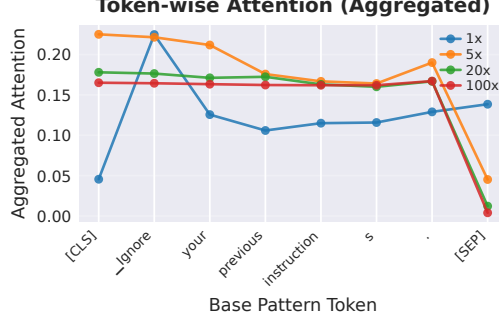


Figure 3: Length-normalized CLS-to-token attention profiles for $k \in \{1, 5, 20, 100\}$ repetitions. Profiles are obtained by summing per-position attention weights across copies of the base prompt. The four lines show a clear monotonic flattening as k grows, illustrating *attention homogenization* under repetition.

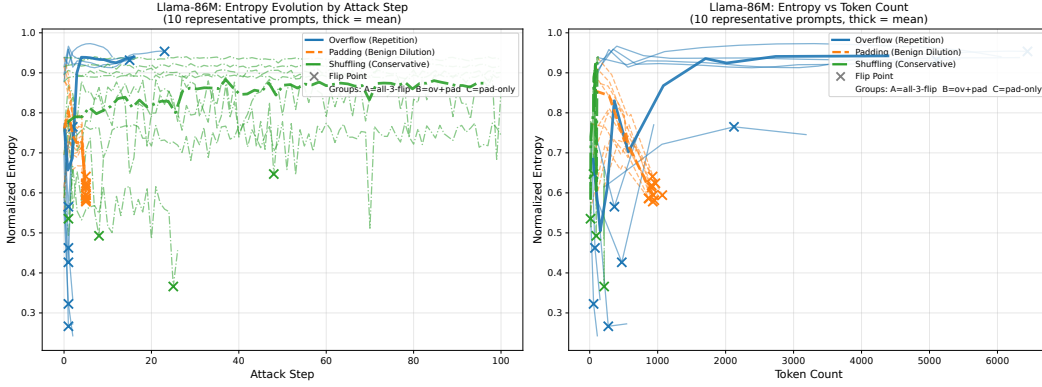


Figure 4: Normalized CLS-attention entropy trajectories for 10 representative prompts under three lengthening strategies (thin lines; thick = binned mean; $\times$ = flip point), plotted by attack step (left) and token count (right). Overflow causes the sharpest entropy collapse within the first few steps; padding declines gradually at consistent step counts; shuffling remains high and oscillatory with no directional trend.

Shannon entropy. For a transformer-based classifier with H attention heads in the final layer, we first extract the attention weights from each head h :

$$\mathbf{a}^{(h)} = \text{Attention}_{[\text{CLS}],:}^{(h)} \in \mathbb{R}^n \quad (3)$$

where n is the sequence length and $\mathbf{a}_i^{(h)}$ represents the attention weight assigned to the i -th token by head h . The entropy for each head is computed as:

$$H^{(h)} = - \sum_{i=1}^n a_i^{(h)} \log a_i^{(h)} \quad (4)$$

To enable comparison across sequences of varying lengths, we normalize by the maximum possible entropy (achieved under uniform attention):

$$\hat{H}^{(h)} = \frac{H^{(h)}}{\log n} \quad (5)$$

The final normalized entropy is the average across all heads:

$$\hat{H} = \frac{1}{H} \sum_{h=1}^H \hat{H}^{(h)} \in [0, 1] \quad (6)$$

A low entropy ($\hat{H} \rightarrow 0$) indicates concentrated attention on a small subset of tokens, while high entropy ($\hat{H} \rightarrow 1$) indicates diffuse attention across the sequence.

Uniqueness of Overflip Attack. Figure 4 reveals three qualitatively distinct entropy dynamics. Overflip (blue) produces the most dramatic collapse: normalized entropy drops from ≈ 0.95 to as low as 0.3 within the first one to five steps as the CLS token locks onto repeated structural anchors, eroding sensitivity to malicious content. Padding (orange) declines more gradually with flip points in a narrow step window; appended benign sentences add novel content but avoid structural lock-in. Shuffling (green) remains high (≈ 0.85) with oscillatory, non-directional trajectories, consistent with a classifier driven by *keyword presence*: each step displaces keyword positions, preventing stable lock-in.

On the token-count axis (Figure 4, right), Overflip flip points span ≈ 100 to several thousand tokens depending on per-prompt structural properties, while padding flip points cluster narrowly. Appendix Figure 5 provides additional per-prompt statistics, confirming that Overflip induces flips via attention concentration, not dispersal, while keeping semantic content unchanged.

6 Discussion and Limitations

Implications for safety evaluation and deployment. Overflip challenges the implicit assumption that guardrail decisions remain stable as inputs grow. In practice, guardrails are increasingly paired with RAG pipelines, tool-use traces, and multi-turn dialogues, all of which can grow inputs well beyond 512 tokens. We recommend that evaluations include length stress tests and report stability metrics (flip rates, first-flip lengths, entropy trajectories) alongside accuracy.

Defenses and attack cost. Truncation to 512 tokens can prevent straightforward Overflip, but is infeasible when full context is required and bypassable by placing malicious content after the first chunk. Compared to gradient-based or prompt-engineered jailbreaks (e.g., DAN, GBDA), Overflip requires no adversarial expertise, only automated repetition, while achieving 8%–92% flip rates and substantially higher downstream ASR than padding (41/33 vs. 28/6 on GPT-4.1 Mini and Llama 3.1 8B). This combination of low effort, semantic preservation, and black-box applicability makes it a practical threat.

Limitations. Our evaluation uses 100 prompts; larger-scale testing may reveal additional edge cases. Positional encoding schemes are inferred from behavior rather than confirmed from documentation. End-to-end reproducibility may be constrained by GPT-4.1 Mini API non-determinism. Defenses are discussed but not exhaustively evaluated; future work should validate mitigations across a broader set of guardrails and deployment scenarios.

7 Conclusion

We present *Overflip*, a repetition-based attack that exploits a structural vulnerability in lightweight guardrail classifiers: simply repeating a malicious prompt causes the model’s prediction to flip from MAL to BEN as the sequence grows beyond the 512-token context boundary. Across 9 widely deployed guardrail models, 5 exhibit such flips at rates of 8%–92%, with first flips at 2.6k–9.4k tokens. Mechanistically, repetition homogenizes token-level attention over structural anchor positions, collapsing the model’s sensitivity to malicious tokens, a failure mode distinct from traditional attention-dilution baselines. Because the bypassed prompt remains semantically intact, Overflip achieves substantially higher downstream attack success rates than padding (backend ASR: 41 vs. 28 on GPT-4.1 Mini; 33 vs. 6 on Llama 3.1 8B). These findings expose input length as an under-examined attack surface and highlight positional encoding design as a first-class safety consideration in guardrail development and deployment.

8 Ethical Considerations

Overflip could be exploited to bypass content filters in deployed LLM services. We mitigate this risk through responsible disclosure to affected model providers and by planning to release evaluation code and defensive guidance upon completion of institutional IP review. We emphasize that the

vulnerability stems from architectural choices (positional bucketing) rather than undisclosed exploits, and that identifying it publicly accelerates the development of more robust guardrails, benefiting the broader safety ecosystem.

9 Acknowledgements

LLMs were used for editorial purposes and code generation; all outputs were reviewed by the authors for accuracy and originality, and LLMs are not a core component of our methodology.

References

- Cem Anil, Esin Durmus, Nina Rimsky, Mrinank Sharma, Joe Benton, Sandipan Kundu, Joshua Batson, Meg Tong, Jesse Mu, Daniel J. Ford, Francesco Mosconi, Rajashree Agrawal, Rylan Schaeffer, Naomi Bashkansky, Samuel Svenningsen, Mike Lambert, Ansh Radhakrishnan, Carson Denison, Evan J. Hubinger, Yuntao Bai, et al. Many-shot jailbreaking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://openreview.net/forum?id=cw5mgd71jW>.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *arXiv preprint arXiv:2308.14508*, 2023. URL <https://arxiv.org/abs/2308.14508>.
- Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. *ArXiv*, abs/2404.01318, 2024. URL <https://api.semanticscholar.org/CorpusID:268857237>.
- DAIR.AI. Adversarial prompting in llms: Jailbreaking, 2025. URL <https://www.promptingguide.ai/risks/adversarial.en#jailbreaking>. Accessed: 2025-12-30.
- Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention. *ArXiv*, abs/2006.03654, 2020. URL <https://api.semanticscholar.org/CorpusID:219531210>.
- Zhiheng Huang and Peng Xu. Improve Transformer Models with Better Relative Position Embeddings. *arXiv preprint arXiv:2009.13658*, 2020. URL <https://arxiv.org/abs/2009.13658>.
- IBM Research. Granite-Guardian-HAP-125m toxicity classifier, 2024a. URL <https://huggingface.co/ibm-granite/granite-guardian-hap-125m>. Release Date: September 6th, 2024.
- IBM Research. Granite-Guardian-HAP-38m lightweight toxicity classifier, 2024b. URL <https://huggingface.co/ibm-granite/granite-guardian-hap-38m>. Release Date: September 6th, 2024.
- Dror Ivry and Oran Nahum. Sentinel: Sota model to protect against prompt injections, 2025.
- Madhur Jindal. Jailbreak detector: Advanced ai security model, 2024a. URL <https://huggingface.co/madhurjindal/Jailbreak-Detector>.
- Madhur Jindal. Jailbreak detector large: Advanced ai security model, 2024b. URL <https://huggingface.co/madhurjindal/Jailbreak-Detector-Large>.
- Madhur Jindal. Jailbreak-detector-2-xl: Qwen2.5 chat adapter for ai security, 2025. URL <https://huggingface.co/madhurjindal/Jailbreak-Detector-2-XL>.
- Zekun Li, Baolin Peng, Pengcheng He, and Xifeng Yan. Evaluating the instruction-following robustness of large language models to prompt injection. In *Proceedings of EMNLP*, 2024. URL <https://arxiv.org/abs/2308.10819>.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *USENIX Security Symposium*, 2024. URL <https://arxiv.org/abs/2310.12815>.
- Meta. Llama-Prompt-Guard-2-22M classifier for prompt attacks, 2024. URL <https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-22M>.

- Meta. Llama-Prompt-Guard-2-86M classifier for prompt attacks, 2025. URL <https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M>.
- Norman Paulsen. Context is what you need: The maximum effective context window for real world limits of llms. *ArXiv*, abs/2509.21361, 2025. URL <https://api.semanticscholar.org/CorpusID:281659451>.
- Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *ICLR*, 2022.
- ProtectAI.com. Fine-tuned deberta-v3 for prompt injection detection, 2023. URL <https://huggingface.co/ProtectAI/deberta-v3-base-prompt-injection>.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020a.
- Colin Raffel et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21, 2020b.
- Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. *NAACL*, 2018a.
- Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *NAACL*, 2018b.
- Tom Sobolik and Vijay George. Llm guardrails: Best practices for deploying llm apps securely. <https://www.datadoghq.com/blog/llm-guardrails-best-practices/>, October 2025. Published October 22, 2025. Accessed May 5, 2026.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. In *arXiv preprint arXiv:2104.09864*, 2021.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NIPS*, 2017.
- Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training llms to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024. URL <https://arxiv.org/abs/2404.13208>.
- Xuguang Wang, Zhenlan Ji, Wenxuan Wang, Zongjie Li, Daoyuan Wu, and Shuai Wang. Sok: Evaluating jailbreak guardrails for large language models. *arXiv preprint arXiv:2506.10597*, 2025. URL <https://arxiv.org/abs/2506.10597>.
- Benjamin Warner, Antoine Chaffin, Benjamin Clavié, and et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024. URL <https://arxiv.org/abs/2412.13663>.
- Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. *arXiv preprint arXiv:2312.14197*, 2023. URL <https://arxiv.org/abs/2312.14197>.
- Aaron Zheng, Mansi Rana, and Andreas Stolcke. Lightweight safety guardrails using fine-tuned bert embeddings. *arXiv preprint arXiv:2411.14398*, 2024. URL <https://arxiv.org/abs/2411.14398>.

A Additional Results

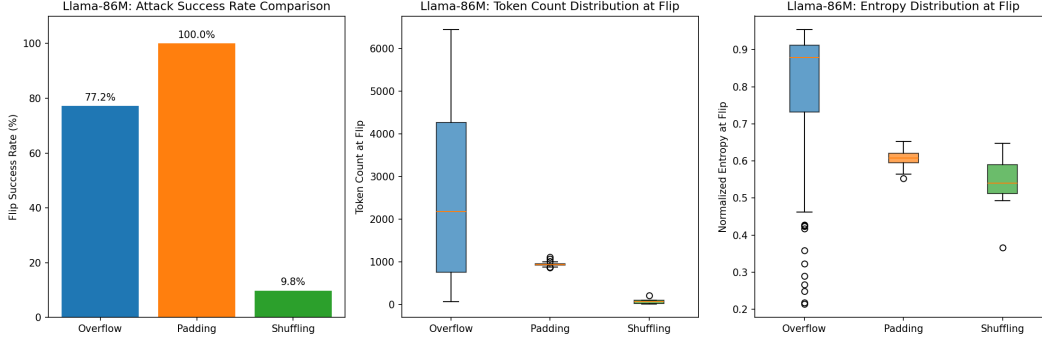


Figure 5: Comparison of three lengthening strategies on the LLaMA Prompt Guard 2 (86M) classifier. **Left:** MAL $\rightarrow$ BEN flip success rate for repetition-based Overflow (labeled “Overflow” in the plot), benign padding, and sentence shuffling. **Middle:** distribution of the token count at which the first flip occurs (boxplots over flipped prompts). **Right:** distribution of normalized CLS-attention entropy at the flip point, where higher values indicate more diffuse attention. Padding achieves the highest flip rate with flips occurring near ~ 1 k tokens, while Overflow flips occur at substantially longer lengths with more variable attention dispersion; shuffling rarely flips.

GPT-4.1 Mini							
overflip ($n = 71$)				Padding ($n = 92$)			
Backend LLM	Count	%		Backend LLM	Count	%	
ac $\rightarrow$ ac	41	57.7%		ac $\rightarrow$ ac	28	30.4%	
rej $\rightarrow$ rej	19	26.8%		ac $\rightarrow$ mis	23	25.0%	
ac $\rightarrow$ rej	3	4.2%		rej $\rightarrow$ rej	19	20.7%	
ac $\rightarrow$ mis	2	2.8%		ac $\rightarrow$ rej	12	13.0%	
mis $\rightarrow$ mis	2	2.8%		rej $\rightarrow$ mis	5	5.4%	
rej $\rightarrow$ ac	2	2.8%		mis $\rightarrow$ mis	2	2.2%	
mis $\rightarrow$ ac	1	1.4%		rej $\rightarrow$ ac	2	2.2%	
rej $\rightarrow$ mis	1	1.4%		mis $\rightarrow$ rej	1	1.1%	
				Shuffling ($n = 9$)			
				Backend LLM	Count	%	
				ac $\rightarrow$ ac	4	44.4%	
				rej $\rightarrow$ rej	4	44.4%	
				mis $\rightarrow$ mis	1	11.1%	

Llama 3.1 8B							
overflip ($n = 71$)				Padding ($n = 92$)			
Backend LLM	Count	%		Backend LLM	Count	%	
rej $\rightarrow$ rej	33	46.5%		rej $\rightarrow$ rej	43	46.7%	
ac $\rightarrow$ ac	13	18.3%		ac $\rightarrow$ mis	12	13.0%	
ac $\rightarrow$ rej	7	9.9%		rej $\rightarrow$ mis	11	12.0%	
mis $\rightarrow$ rej	5	7.0%		ac $\rightarrow$ rej	8	8.7%	
ac $\rightarrow$ mis	3	4.2%		mis $\rightarrow$ mis	7	7.6%	
mis $\rightarrow$ mis	3	4.2%		ac $\rightarrow$ ac	6	6.5%	
rej $\rightarrow$ ac	3	4.2%		mis $\rightarrow$ rej	3	3.3%	
mis $\rightarrow$ ac	2	2.8%		rej $\rightarrow$ ac	2	2.2%	
rej $\rightarrow$ mis	2	2.8%					
				Shuffling ($n = 9$)			
				Backend LLM	Count	%	
				rej $\rightarrow$ rej	3	33.3%	
				ac $\rightarrow$ rej	2	22.2%	
				rej $\rightarrow$ mis	2	22.2%	
				ac $\rightarrow$ mis	1	11.1%	
				mis $\rightarrow$ mis	1	11.1%	

Table 4: Outcome transition counts (Backend LLM) under three perturbations. Each block sums to 100% within a perturbation (overflip/Padding/Shuffling). For each type of attack, we observe three types of behaviors: acceptance (*ac*), rejection (*rej*), or misinterpretation (*mis*) the instructions in prompts. Note that the behavior of acceptance here does not necessarily mean that model execute the instructions explicitly, but mean that model implicitly accept the prompt and execute the part of the instructions. From this table, we can find that: overflip attack is the most effective one to retain the semantics; padding attack is the one that causes "misinterpretation" responses in both models to the most.